

Kotlinは、クラスやオブジェクトを用いたオブジェクト指向プログラミングをサポートしています。オブジェクトは、プログラム内でデータを格納するのに役立ちます。クラスを使用すると、オブジェクトの一連の特性を定義することができます。クラスからオブジェクトを作成する場合、その特性を毎回宣言する必要がないため、時間と手間を節約できます。

クラスを宣言するには、`class` キーワードを使用します：

```
class Customer
```

Properties

プロパティ

クラスのオブジェクトの特性は、プロパティとして宣言することができます。クラスに対してプロパティを宣言するには、次のようにします。

- クラス名の後の括弧 `()` の中での宣言。

```
class Contact(val id: Int, var email: String)
```

- 中括弧 `{}` で囲まれたクラス本体内での宣言。

```
class Contact(val id: Int, var email: String) {  
    val category: String = ""  
}
```

クラスのインスタンスが作成された後に変更する必要がない限り、プロパティは読み取り専用 (`val`) として宣言することをお勧めします。

括弧内に `val` や `var` を指定せずにプロパティを宣言することは可能ですが、インスタンスが作成された後は、これらのプロパティにアクセスすることはできません。

- 括弧 `()` 内に含まれる内容は、**クラスヘッダー**と呼ばれます。
- クラスのプロパティを宣言する際は、**末尾のコンマ**を使用できます。

{style="note"}

関数の引数と同様に、クラスのプロパティにもデフォルト値を指定することができます：

```
class Contact(val id: Int, var email: String = "example@gmail.com") {  
    val category: String = "work"  
}
```

Create instance

インスタンスの作成

クラスからオブジェクトを作成するには、**コンストラクタ**を使用してクラスの**インスタンス**を宣言します。

デフォルトでは、Kotlin はクラスヘッダーで宣言されたパラメータを持つコンストラクタを自動的に生成します。

例えば：

```
class Contact(val id: Int, var email: String)

fun main() {
    val contact = Contact(1, "mary@gmail.com")
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-class-create-instance"}
```

この例では：

- `Contact` はクラスです。
- `contact` は `Contact` クラスのインスタンスです。
- `id` と `email` はプロパティです。
- `id` と `email` は、デフォルトのコンストラクタとともに `contact` を作成するために使用されます。

Kotlinのクラスには、自分で定義したものを含め、複数のコンストラクタを定義することができます。複数のコンストラクタの宣言方法について詳しくは、[コンストラクタ](#)を参照してください。

Access properties

プロパティへのアクセス

インスタンスのプロパティにアクセスするには、インスタンス名の後にピリオド `.` を付けてプロパティ名を記述します：

```
class Contact(val id: Int, var email: String)

fun main() {
    val contact = Contact(1, "mary@gmail.com")

    // プロパティ「email」の値を出力します
    println(contact.email)
    // mary@gmail.com

    // プロパティ「email」の値を更新します
    contact.email = "jane@gmail.com"

    // プロパティ「email」の新しい値を出力します
    println(contact.email)
}
```

```
// jane@gmail.com  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-access-property"}
```

プロパティの値を一つの文字列の一部として連結するには、文字列テンプレート（\$）を使用できます。例えば：

```
println("Their email address is: ${contact.email}")
```

```
{style="tip"}
```

Member functions

メンバー関数

プロパティをオブジェクトの特性として宣言するだけでなく、メンバー関数を使ってオブジェクトの挙動を定義することもできます。

Kotlin では、メンバー関数はクラス本体内で宣言する必要があります。インスタンスのメンバー関数を呼び出すには、インスタンス名の後にピリオド `.` を付けて関数名を記述します。例えば：

```
class Contact(val id: Int, var email: String) {  
    fun printId() {  
        println(id)  
    }  
}  
  
fun main() {  
    val contact = Contact(1, "mary@gmail.com")  
    // メンバ関数 printId() を呼び出します  
    contact.printId()    // 1  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-member-function"}
```

Data classes

データクラス

Kotlinには、データの保存に特に役立つ**データクラス**があります。データクラスはクラスと同じ機能を持っていますが、追加のメンバー関数が自動的に備わっています。これらのメンバー関数を使用すると、インスタンスを読みやすい形式で出力したり、クラスのインスタンス同士を比較したり、インスタンスをコピーしたりすることが簡単にできます。これらの関数は自動的に利用可能なので、クラスごとに同じ定型コードを書く手間が省けます。

データクラスを宣言するには、キーワード `data` を使用します：

```
data class User(val name: String, val id: Int)
```

Kotlin コンパイラは、メンバ関数を生成する際、[プライマリコンストラクタ](#) 内で定義されたプロパティのみを使用します。データクラスの本体内でプロパティを宣言した場合、それらは生成された関数の出力には含まれません。

データクラスにおいて、最も有用な定義済みメンバ関数は以下の通りです：

関数名	説明
<code>toString()</code>	クラスインスタンスとそのプロパティについて、読みやすい文字列を出力します。
<code>equals()</code> or <code>==</code>	あるクラスのインスタンス同士を比較します。
<code>copy()</code>	別のクラスインスタンスをコピーして、場合によってはプロパティを一部変更して、新しいクラスインスタンスを作成します。

各関数の使用例については、以下のセクションを参照してください：

- [文字列として出力](#)
- [インスタンスの比較](#)
- [インスタンスのコピー](#)

Print as string

文字列として出力

クラスインスタンスの読みやすい文字列を出力するには、`toString()` 関数を明示的に呼び出するか、自動的に `toString()` を呼び出してくれる出力関数（`println()` や `print()`）を使用することができます。

```
data class User(val name: String, val id: Int)

fun main() {
    //sampleStart
    val user = User("Alex", 1)

    // 出力が読みやすいように、自動的に toString() 関数が使用されます
    println(user)          // User(name=Alex, id=1)
    //sampleEnd
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-data-classes-print-string"}
```

これは、デバッグやログ作成の際に特に役立ちます。

Compare instances

インスタンスの比較

データクラスのインスタンスを比較するには、等号演算子 `==` を使用します：

```
data class User(val name: String, val id: Int)

fun main() {
    //sampleStart
    val user = User("Alex", 1)
    val secondUser = User("Alex", 1)
    val thirdUser = User("Max", 2)

    // secondUserとの比較
    println("user == secondUser: ${user == secondUser}")
    // user == secondUser: true

    // thirdUserとの比較
    println("user == thirdUser: ${user == thirdUser}")
    // user == thirdUser: false
    //sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-data-classes-compare-instances"}

Copy instance

インスタンスをコピー

データクラスのインスタンスの完全なコピーを作成するには、そのインスタンスに対して `copy()` 関数を呼び出します。

データクラスのインスタンスのコピーを作成し、**かつ**一部のプロパティを変更するには、そのインスタンスに対して `copy()` 関数を呼び出し、**かつ**関数の引数としてプロパティの置き換え用値を指定します。

例えば：

```
data class User(val name: String, val id: Int)

fun main() {
    //sampleStart
    val user = User("Alex", 1)

    // ユーザーの完全なコピーを作成します
    println(user.copy())
    // User(name=Alex, id=1)

    // 名前が「Max」のユーザーのコピーを作成します
    println(user.copy("Max"))
    // User(name=Max, id=1)

    // IDが3のユーザーのコピーを作成します
    println(user.copy(id = 3))
}
```

```
// User(name=Alex, id=3)
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-data-classes-copy-instance"}

インスタンスのコピーを作成することは、元のインスタンスを変更するよりも安全です。なぜなら、元のインスタンスに依存しているコードは、そのコピーやそれに対して行う操作の影響を受けないからです。

データクラスに関する詳細については、[データクラス](#)を参照してください。

このツアーの最後の章では、Kotlinの[null安全性](#)について解説します。

演習 {completion-point="true"}

課題 1 {initial-collapse-state="collapsed" collapsible="true"}

2つのプロパティ（名前用と給与用）を持つデータクラス `Employee` を定義してください。「給与」のプロパティが変更可能になっていることを確認してください。そうしないと、年末に給与アップが適用されません！メイン関数では、このデータクラスの使用法を示しています。

|---|---|

```
// Write your code here

fun main() {
    val emp = Employee("Mary", 20)
    println(emp)
    emp.salary += 10
    println(emp)
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-classes-exercise-1"}

|---|---|

```
data class Employee(val name: String, var salary: Int)

fun main() {
    val emp = Employee("Mary", 20)
    println(emp)
    emp.salary += 10
    println(emp)
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-classes-solution-1"}

課題 2 {initial-collapse-state="collapsed" collapsible="true"}

このコードをコンパイルするために必要な追加のデータクラスを宣言してください。

|---|---|

```
data class Person(val name: Name, val address: Address, val ownsAPet: Boolean = true)
// Write your code here
// data class Name(...)

fun main() {
    val person = Person(
        Name("John", "Smith"),
        Address("123 Fake Street", City("Springfield", "US")),
        ownsAPet = false
    )
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-classes-exercise-2"}

|---|---|

```
data class Person(val name: Name, val address: Address, val ownsAPet: Boolean = true)
data class Name(val first: String, val last: String)
data class Address(val street: String, val city: City)
data class City(val name: String, val countryCode: String)

fun main() {
    val person = Person(
        Name("John", "Smith"),
        Address("123 Fake Street", City("Springfield", "US")),
        ownsAPet = false
    )
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-classes-solution-2"}

課題 3 {initial-collapse-state="collapsed" collapsible="true"}

コードをテストするには、ランダムな従業員を生成できるジェネレータが必要です。候補となる名前の固定リスト（クラス本体内部）を持つ `RandomEmployeeGenerator` クラスを定義してください。クラスに最低給与額と最高給与額を設定します（クラスヘッダー内）。クラスの本体内部で、`generateEmployee()` 関数を定義してください。ここでも、メイン関数がこのクラスの使い方を示しています。

この演習では、`Random.nextInt()` 関数を使用できるように、パッケージをインポートします。パッケージのインポートに関する詳細については、[パッケージとインポート](#)を参照してください。

{style="tip"}

Lists have an extension function called `.random()` that returns a random item within a list.

`Random.nextInt(from = ..., until = ...)` gives you a random `Int` number within specified limits.

|---|---|

```
import kotlin.random.Random

data class Employee(val name: String, var salary: Int)

// Write your code here

fun main() {
    val empGen = RandomEmployeeGenerator(10, 30)
    println(empGen.generateEmployee())
    println(empGen.generateEmployee())
    println(empGen.generateEmployee())
    empGen.minSalary = 50
    empGen.maxSalary = 100
    println(empGen.generateEmployee())
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-classes-exercise-3"}

|---|---|

```
import kotlin.random.Random

data class Employee(val name: String, var salary: Int)

class RandomEmployeeGenerator(var minSalary: Int, var maxSalary: Int) {
    val names = listOf("John", "Mary", "Ann", "Paul", "Jack", "Elizabeth")
    fun generateEmployee() =
        Employee(names.random(),
            Random.nextInt(from = minSalary, until = maxSalary))
}

fun main() {
    val empGen = RandomEmployeeGenerator(10, 30)
    println(empGen.generateEmployee())
    println(empGen.generateEmployee())
    println(empGen.generateEmployee())
    empGen.minSalary = 50
    empGen.maxSalary = 100
}
```

```
println(empGen.generateEmployee())  
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-classes-solution-3"}

- [Previous step](#)
- [Next step](#)